

Chapter 1

Assignment 2: Semantic Extraction

Building on the syntactic pipeline of Assignment 1, this stage extracts *semantic* information from each clause: named entities that identify the parties, values, and legal references involved; semantic roles that describe who does what to whom; and an intent label that captures the functional type of the clause.

1.1 Custom Named Entity Recognition (NER)

1.1.1 Entity Schema

Six domain-specific entity types are defined for legal contract NER:

- **PARTY** contracting parties and organisations (e.g. *Party A*, *ABC International Co., Ltd.*).
- **MONEY** monetary amounts with or without currency symbols (e.g. *USD 10,000*, *\$5,000*).
- **DATE** absolute and relative date expressions (e.g. *05 May 2024*, *within 30 days*).
- **RATE** percentage or per-unit rates (e.g. *1% per month*, *2% per annum*).
- **PENALTY** penalty clauses and liquidated-damages expressions.
- **LAW** statutory references, ordinances, and legal codes (e.g. *Civil Code 2015*).

The full BIO label set used during model training is:

O, B-PARTY, I-PARTY, B-MONEY, I-MONEY, B-DATE, I-DATE, B-RATE, I-RATE,
B-PENALTY, I-PENALTY, B-LAW, I-LAW

1.1.2 Model Architecture

The system adopts a *hybrid* design in which a deterministic rule-based extractor and a fine-tuned Legal-BERT model are executed in parallel for every clause and their outputs are merged into a single entity list.

Primary system Rule-based NER. The RuleBasedNER module applies hand-crafted regular expressions tailored to each of the six entity types. Representative patterns include monetary expressions such as `(USD|VND|EUR|GBP|AUD)\s*[\d,\.]+` and `\$\s*[\d,\.]+` for MONEY; ISO-format, ordinal, natural-language, and parenthesised duration patterns (e.g. *1st day of March, 2024, thirty (30) Business Days, within 30 days*) for DATE; case-sensitive role nouns (*Employer, Lessor, Service Provider, ...*) and *Party A/B* for PARTY; statutory references such as *Law No. 50/2005/QH11, Decree No. 13/2023/ND-CP, Civil Code*, and *SIAC Rules* for LAW; and keywords such as *penalty, liquidated damages, termination fee* for PENALTY. Because regex matches on legal contract text are highly precise, rule-based predictions are treated as the primary system and take precedence during merging.

Supplementary system Legal-BERT fine-tuned NER. A token-classification head is fine-tuned on top of `nlpaueb/legal-bert-base-uncased` [1] using the HuggingFace `Trainer` API. Training data combines a small hand-annotated set (`data/english/sample_ner.txt`) with *silver labels* auto-generated by running the rule-based extractor over every clause produced by Assignment 1. This bootstrapping enables Legal-BERT to learn entity boundaries on PARTY mentions and long multi-token spans that the regexes do not cover.

Silver-label denoising. A noticeable failure mode of the first iteration was that Legal-BERT, after training on raw silver labels, produced spurious predictions such as “*of*” $\rightarrow$ *RATE* and “*)*” $\rightarrow$ *RATE* at inference time. These arose because occasional regex over-matches were being treated as ground truth during fine-tuning. Two filters were therefore added (`_is_valid_silver_entity`): (i) reject entity spans of fewer than two characters, (ii) reject spans that contain no alphanumeric characters, and (iii) reject spans whose tokens are all closed-class stopwords (e.g. *of, the, a, to, and, or*). The same filter is reapplied to Legal-BERT predictions at inference, as a defence-in-depth against any residual noise. Key training hyperparameters are summarised in Table 1.1.

Table 1.1: Legal-BERT NER training configuration.

Hyperparameter	Value
Base model	<code>nlpaueb/legal-bert-base-uncased</code>
Epochs	5
Learning rate	2×10^{-5}
Batch size	8 (train and eval)
Weight decay	0.01
Max sequence length	512 tokens
Train/eval split	90 / 10 (<code>seed=42</code>)
Eval / save strategy	per epoch (best model loaded)
Evaluation metric	sequeval (entity-level P/R/F1)
Training data	manual + rule-based silver labels

Prediction merging. At inference time, the rule-based and Legal-BERT predictions are combined by `_merge_entities`: all rule-based spans are kept first, and a BERT-predicted span is appended only if it does not overlap any existing span. This guarantees

the deterministic regex precision is never overwritten while still allowing Legal-BERT to contribute additional entities particularly PARTY mentions that fall outside the regex vocabulary. If the fine-tuned checkpoint at `models/ner_legal_bert_v2` cannot be loaded, the pipeline gracefully degrades to rule-based predictions only.

1.1.3 Output Format

Results are written to `output/ner_results.json` as a list of objects:

```

1  [
2    {
3      "clause_id": 0,
4      "text": "Party A shall pay Party B USD 10,000 by 05 May
5             2024...",
6      "entities": [
7        {"text": "Party A",      "label": "PARTY", "start": 0, "
8          end": 7},
9        {"text": "Party B",      "label": "PARTY", "start": 16, "
10         end": 23},
11       {"text": "USD 10,000",    "label": "MONEY", "start": 24, "
12         end": 34},
13       {"text": "05 May 2024",  "label": "DATE",   "start": 38, "
14         end": 49},
15       {"text": "1% per month", "label": "RATE",   "start": 53, "
16         end": 65}
17     ]
18   }
19 ]

```

Listing 1.1: NER output format (abbreviated).

1.1.4 NER Example

Table 1.2 shows the entities extracted from a representative legal clause.

Table 1.2: Entities extracted from “*Party A shall pay Party B USD 10,000 by 05 May 2024 at 1% per month penalty rate.*”

Entity Text	Label	Start	End
Party A	PARTY	0	7
Party B	PARTY	16	23
USD 10,000	MONEY	24	34
05 May 2024	DATE	38	49
1% per month	RATE	53	65

1.2 Semantic Role Labelling (SRL)

1.2.1 Approach

Rather than a dedicated BERT-based SRL model, the system uses a *dependency-tree-based* approach built on top of spaCy’s `en_core_web_sm` parser. For each clause, the ROOT token (typically the main verb) is identified as the predicate, and semantic roles are assigned to its dependents by mapping Universal Dependencies labels to PropBank-style argument roles, as shown in Table 1.3.

Table 1.3: Mapping from Universal Dependencies labels to semantic roles.

UD Dependency Label	Semantic Role	Notes
nsubj	Agent	Active-voice subject
agent + pobj	Agent	Passive “by” phrase
nsubjpass	Theme	Passive-voice subject
dobj / obj	Theme	Direct object
iobj	Recipient	Indirect object
prep (<i>to</i>)	Recipient	Prepositional recipient
prep (temporal)	Time	See list below
advcl + cond. mark	Condition	Subordinate conditional clause

Temporal prepositions and condition markers. A prepositional phrase headed by the ROOT verb is classified as Time when its preposition belongs to the set $\{before, after, on, within, by, until, upon, during\}$. An adverbial clause (`advcl`) attached to the ROOT is classified as Condition when its subordinating marker (`mark`) is one of $\{if, unless, provided, subject\}$. After the first pass, two fallback sweeps over the ROOT’s children fill in any Recipient or Time slot that was missed because the relevant preposition was attached at a deeper level.

Clause filtering. The contract corpus produced by Assignment 1 contains many fragments that are not full sentences: section headings (“*ARTICLE 1 DEFINITIONS*”), document titles (“*SOFTWARE SERVICES AGREEMENT*”), and short appositive definitions of parties (“*TechVision Solutions Pte. Ltd.*”). Running spaCy on these fragments yields a noun ROOT and a meaningless “predicate” such as *AGREEMENT* or *Ltd.* together with empty role assignments. Two preprocessing checks are therefore applied: `_is_processable` rejects all-uppercase fragments and clauses shorter than five characters, and the SRL extractor additionally requires at least one VERB or AUX token in the parsed document. Clauses failing either check are returned with empty roles and an empty predicate.

1.2.2 Output Format

Results are written to `output/srl_results.json` as a list of objects with the following structure:

```
1 [
2   {
```

```

3      "clause_id": 0,
4      "text": "The Employer shall pay the Employee the monthly
           salary ...",
5      "predicate": "pay",
6      "roles": {
7          "Agent":      "The Employer",
8          "Theme":      "the monthly salary",
9          "Recipient":  "the Employee",
10         "Time":       "within 30 days"
11     }
12 }
13 ]

```

Listing 1.2: SRL output format.

1.2.3 SRL Example

Input:

“The Employer shall pay the Employee the monthly salary within 30 days.”

Output:

Predicate: *pay*
 Agent: *The Employer*
 Theme: *the monthly salary*
 Recipient: *the Employee*
 Time: *within 30 days*

The ROOT token *pay* is the predicate. *The Employer* is the active nominal subject (nsubj) mapped to Agent; *the monthly salary* is the direct object (dobj) mapped to Theme; *the Employee* is reached via the indirect object (iobj) relation mapped to Recipient; and *within 30 days* is a temporal prepositional phrase mapped to Time.

1.3 Intent Classification

1.3.1 Task Definition and Label Set

Intent classification assigns each clause one of four functional intent labels. For each label, a curated list of regular-expression keyword patterns (compiled with `re.IGNORECASE`) is used by the weak labeller.

- **Obligation** a party is required to perform an action. Patterns: *shall* (without *not*), *must* (without *not*), *is/are required to*, *is obligated to*, *hereby agrees to*, *undertakes to*, *will be responsible*.
- **Prohibition** an action is forbidden. Patterns: *shall not*, *must not*, *is/are prohibited*, *is forbidden*, *not permitted*, *no Party shall*, *shall refrain*.
- **Right** a party is entitled to perform an action. Patterns: *may*, *is/are entitled to*, *has/have the right*, *can*, *at its/their discretion*, *has the option*.

- **Termination Condition** the contract terminates upon an event. Patterns: *terminat...*, *expir...*, *upon breach*, *in the event of default/breach/non-payment*, *shall be void*, *null and void*, *early termination*, *withdrawn*, *dissolution*.

Because several patterns can co-occur within a single clause (e.g. a *shall not* prohibition that also mentions *terminate*), a priority order **Prohibition** > **Termination Condition** > **Right** > **Obligation** is enforced during weak labelling: the highest-priority intent whose pattern matches is selected. Clauses that match no pattern are labelled **Unknown** and removed before supervised training.

1.3.2 Model 1 TF-IDF + Logistic Regression

Training labels are generated by the keyword-regex weak labeller described above. Clauses labelled **Unknown** are discarded, and the remaining labelled examples are used to fit a scikit-learn **Pipeline** that chains a TF-IDF vectoriser to a multinomial Logistic Regression classifier. Table 1.4 summarises the configuration.

Table 1.4: TF-IDF + Logistic Regression configuration.

Component	Setting
<i>TfidfVectorizer</i>	
n-gram range	(1, 3)
max features	5 000
sublinear TF	True
accent stripping	unicode
token pattern	\b[a-zA-Z][a-zA-Z0-9-']*\\b
<i>LogisticRegression</i>	
Regularisation C	1.0
Solver	lbfgs
Max iterations	1 000
Class weight	balanced

1.3.3 Model 2 Fine-tuned BERT

The second classifier fine-tunes **bert-base-uncased** as a sequence classifier using **AutoModelForSequenceClassification**. At inference time, a softmax over the logits yields a confidence score that is reported alongside the predicted label.

Class-imbalance handling. On the contract corpus, the weak labeller produces a heavily skewed distribution: *Obligation* clauses dominate ($\sim 83\%$) while *Right* and *Termination Condition* together account for less than 5% of the labelled set. A plain cross-entropy loss therefore drives the model to a trivial majority-class predictor that ignores the minority intents entirely. To counter this, training uses a class-weighted cross-entropy loss

$$\mathcal{L} = - \sum_i w_{y_i} \log p_{\theta}(y_i \mid x_i), \quad w_c = \frac{N}{C \cdot n_c}$$

where N is the total number of labelled clauses, C the number of classes, and n_c the count of class c . The weights are computed once from the training labels and registered on the same device as the model inside a custom `_WeightedTrainer` subclass that overrides `compute_loss`. In addition, an `EarlyStoppingCallback(patience=2)` watches the macro-F1 on the held-out split, and the best checkpoint is restored at the end of training. Table 1.5 summarises the resulting training settings.

Table 1.5: BERT intent classification training configuration.

Hyperparameter	Value
Base model	<code>bert-base-uncased</code>
Epochs	8 (with early stopping, patience 2)
Learning rate	2×10^{-5}
Batch size	8 (train and eval)
Weight decay	0.01
Max token length	256
Train/eval split	90 / 10 (<code>seed=42</code>)
Eval / save strategy	per epoch (best model loaded)
Best-model metric	macro-F1 (higher is better)
Loss function	class-weighted cross-entropy
Evaluation metric	accuracy, macro-F1

1.3.4 Output Format

Results are written to `intent_classification.txt` as a TSV file with columns `clause_text`, `tfidf_label`, `bert_label`, and `confidence`.

1.3.5 Comparative Example

Table 1.6 compares the predictions of both models on representative clauses.

Table 1.6: Intent classification results for sample clauses.

Clause Excerpt	TF-IDF	BERT	Conf.
<i>“... shall pay...”</i>	Obligation	Obligation	0.96
<i>“... shall not disclose...”</i>	Prohibition	Prohibition	0.98
<i>“... may terminate...”</i>	Right	Termination condition	0.72
<i>“... terminates upon breach...”</i>	Termination condition	Termination condition	0.95

The “*may terminate*” clause illustrates a common ambiguity: TF-IDF’s n-gram features weight *may* heavily toward the Right class, while BERT’s contextual representation captures the termination semantics of the full phrase and predicts Termination Condition. The lower confidence score (0.72) correctly signals the model’s uncertainty on this boundary case.

1.4 Results on the Contract Corpus

This section reports the actual outputs of the pipeline on the 170 clauses produced by Assignment 1 from a Software Services Agreement. These numbers were obtained after applying the silver-label denoising, SRL clause filtering, and class-weighted loss described in the preceding sections.

1.4.1 NER results

The hybrid extractor identified **249 entities across 114 of 170 clauses** (67% clause coverage). Crucially, the spurious single-token spans that appeared in the first iteration (“of” → *RATE*, “)” → *RATE*) are now *entirely absent*, confirming that the `_is_valid_silver_entity` filter is effective both during training-set construction and at inference time. The entity-type distribution is summarised in Table 1.7.

Table 1.7: NER entity-type distribution on the contract corpus.

Label	Count	Representative span
PARTY	171	<i>Party A, Service Provider, the Parties</i>
DATE	35	<i>1st day of March, 2024, within thirty (30) days</i>
LAW	17	<i>Law No. 50/2005/QH11, Circular No. 05/2017/TT-BTC</i>
MONEY	14	<i>\$150,000 USD, \$2,000,000 USD</i>
PENALTY	10	<i>late payment penalty, liquidated damages</i>
RATE	2	<i>1.5%</i>
Total	249	

The PARTY count dominates because contracting parties are referenced in almost every operational clause, which matches the expected linguistic profile of a bilateral services agreement. The relatively low RATE count reflects that this particular contract contains only one explicit interest-rate clause (*1.5% per month*); the rule-based pattern is correctly conservative and does not match unrelated percentages from text such as “100% ownership”.

1.4.2 SRL results

After clause-level preprocessing, 36 clauses (21%) are correctly classified as non-sentential fragments (titles, article headers, bare entity references) and receive an empty predicate. The remaining 134 clauses produce a verb predicate; of these, 122 yield at least one extracted role, and only 12 verbal clauses receive an empty role set. The aggregate role distribution is shown in Table 1.8.

Table 1.8: SRL role distribution on the contract corpus (134 verbal clauses).

Role	Count
Agent	92
Theme	90
Recipient	18
Condition	19
Time	14

No predicate is now incorrectly assigned to a noun token: previous spurious predicates such as *AGREEMENT*, *Ltd.*, *Corp.*, and *definition* no longer appear.

1.4.3 Intent classification results

Class-aware training has clearly broadened the predicted label distribution compared with the first iteration, in which BERT collapsed to predicting *Obligation* on 166 of 170 clauses. The updated label counts produced by the two models are reported in Table 1.9.

Table 1.9: Predicted intent distribution (170 clauses).

Label	TF-IDF	BERT
Obligation	113	99
Prohibition	29	3
Right	15	5
Termination Condition	13	63

TF-IDF + class-balanced LR is the stronger model on this corpus. The class-weighted Logistic Regression produces a distribution that is consistent with manual inspection: *Obligation* clauses dominate but the minority intents (*Right*, *Termination*, *Prohibition*) are no longer crushed to single-digit counts.

BERT remains under-confident. Although BERT now predicts all four classes, the mean softmax confidence across the corpus is only 0.355 and **every** clause receives a confidence below 0.5. Manual inspection shows that BERT mislabels several clearly obligatory clauses as *Termination Condition* (63 BERT predictions versus only 13 from TF-IDF for this label). Two factors explain this behaviour and are not deficiencies of the model itself:

- **Tiny training set.** After discarding **Unknown** clauses, the supervised training set contains only ~ 150 examples. Fine-tuning a 110M-parameter BERT model on this volume typically requires either external augmentation or upstream domain pre-training.
- **Severe class imbalance.** Even after weighting, the training signal for *Right* (3 raw examples) and *Termination Condition* (3 raw examples) is too weak for stable representation learning.

For deployment on this corpus, the recommendation is therefore to use the TF-IDF model as the primary classifier and to consult BERT only as a soft tie-breaker when the TF-IDF prediction is near-uniform across classes.

Bibliography

- [1] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, *LEGAL-BERT: The Muppets straight out of Law School*, in *Findings of EMNLP 2020*, pp. 2898–2904, 2020.